

1. Database Overview

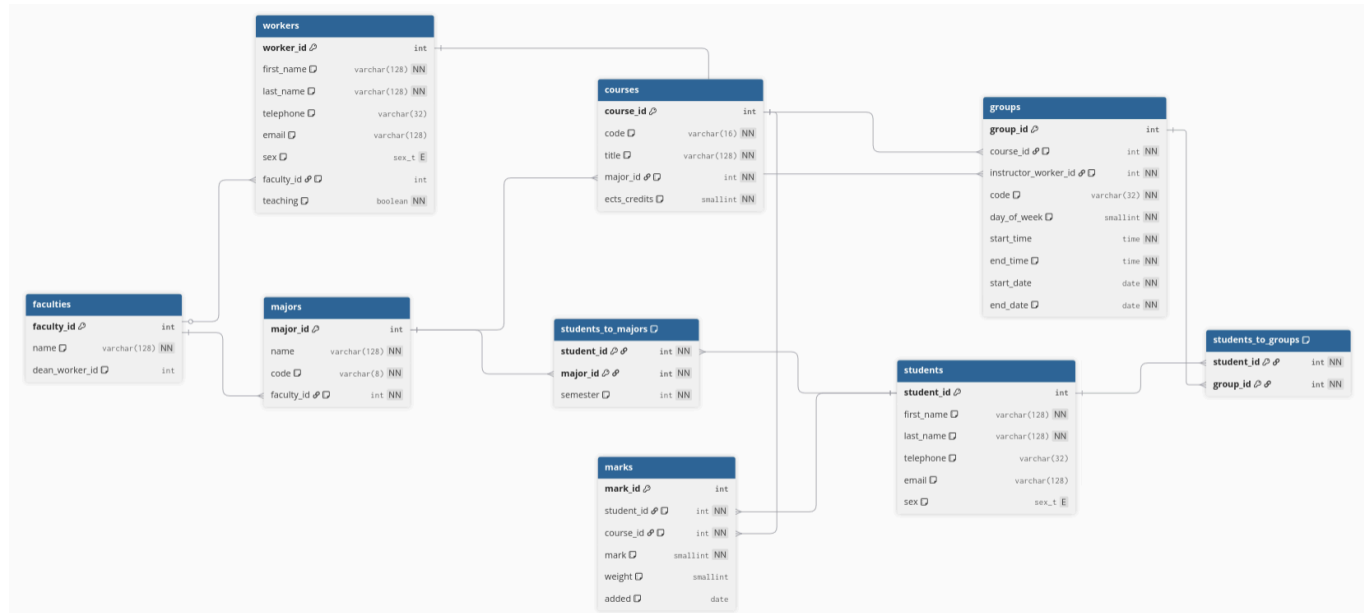
Authors: Maksymilian Oliwa & Piotr Pazdan

This project implements a relational database designed to manage university administrative data. The primary function of the database is to maintain the academic records and enforce rules that prevent common errors.

Problems We Aimed to Solve:

- **Scheduling Conflict Prevention** - The most important feature of the system is the prevention of time overlaps. The database enforces logic that prevents the assignment of a student or instructor to two classes happening at the same time.
- **Data Standardization** - Database implements a basic data normalization (e.g. normalizing phone numbers to E.164 format). This makes working with the database easier and more flexible.
- **Enrollment-Grade Integrity** - Database resolves the issue of "orphaned" grades by strictly enforcing that marks can only be assigned to students enrolled in the corresponding course.

Database Schema



2. Database Structure

The data model is divided into four logical components: Organization, People, Scheduling and Grading.

- **Organization Hierarchy** - The university structure consists of `faculties`, `majors`, and `courses` tables. These tables utilize cascading foreign keys to ensure integrity; if a parent

entry (such as a Major) is removed, dependent entities (Courses) are automatically cleaned up.

- People Management - Data is stored in `students` and `workers` tables including personal information such as phone numbers and email addresses. The `workers` table includes a boolean `teaching` attribute, which distinguishes between administrative staff and academic instructors.
- Scheduling and Enrollment - The execution of the curriculum is managed by the `groups` table, which defines specific days, time slots, and semester date ranges. To support academic requirements, the database uses many-to-many junction tables. The `students_to_majors` and `students_to_groups` tables allow for the assignment of a single student to multiple classes and majors.
- Grading - Student performance is tracked via the `marks` table. Data quality is provided using CHECK constraints that restrict input values to the university grading scale (2.0, 3.0, 3.5, 4.0, 4.5, 5.0).

3. Domains

Custom data types (Domains) were implemented to enforce various validation rules:

- `email_t` - Enforces standard email syntax via Regular Expressions.
- `telephone_t` - Accepts variable inputs and is paired with triggers to standardize storage.
- `name_t` - Restricts personal name inputs to valid alphabetic characters.

4. Triggers

PL/pgSQL triggers are used to automate logic and validate constraints that go beyond simple data types.

- **Overlap Detection** - The triggers `trg_group_assignment` and `trg_instructor_overlap` execute before any schedule modification. They compare the proposed time slot (`start_time`, `end_time`, `start_date`, `end_date`) against all existing records for the specific person. If a collision is detected, the transaction is rejected.
- **Data Sanitization** - Triggers automatically format data upon insertion. The `normalize_phone` function strips formatting and applies the E.164 standard, while `capitalize_name` ensures the capitalization of personal names.
- **Dynamic Status Updates** - The `trg_instructor_teaching` trigger monitors the `groups` table. It automatically updates the `teaching` status in the `workers` table to true or false depending on whether the worker is currently assigned to any groups:

5. Views and Functions

Several database objects were created to simplify data retrieval for end-users:

- **GPA Calculation** - The `students_gpa` view dynamically calculates the weighted average grade for students, grouped by major.
- **Schedule Transformation** - The functions `student_schedule` and `instructor_schedule` transform row-based scheduling data into a weekly calendar format (Monday through Friday), simplifying timetable generation.
- **Workload Analysis** - The `worker_overview` view aggregates total teaching hours per semester to assist in faculty load management.
- **Student Analysis** - The `student_overview` aggregates various information about students including their workload and grades. It can be useful for monitoring academic progress.

6. Access Control

Security is managed via a Role-Based Access Control model with three levels:

- University Admin - Grants full privileges for schema modification and user management.
- Instructor - Grants read access to course and student lists. Write access is restricted exclusively to the `marks` table. Some delete permissions are revoked to prevent data loss.
- Student - Grants read-only access to the public course catalog (`courses` , `majors`) and limited staff directory information. Access to private data of other students is restricted.

7. Performance Optimization

To improve system responsiveness, particularly during complex validation checks, indexing was applied:

- Trigger Acceleration - A composite index `idx_groups_dates` (covering `day_of_week` , `start_date` , and `end_date`) was implemented on the `groups` table. It was added to reduce the computational cost of the `trg_group_assignment` overlap detection, which must frequently scan for conflicting date ranges.
- Linking Tables: Explicit indexes were created on foreign key columns (e.g., `idx_fk_majors_faculty` , `idx_fk_courses_major`). They were intended to optimize the performance of `JOIN` operations in views and speed up scans during cascading delete or update operations.

8. Administrative Procedures

Beyond the core schema, the database includes logic that simplifies administrative tasks:

- Smart Enrollment - The `enroll_student_in_major` function provides an abstraction layer for student registration. It allows administrators to use natural keys (Student Email and Major Code) rather than internal IDs. The function utilizes `ON CONFLICT` logic to automatically differentiate between a new enrollment and a semester update for an existing student.
- Hierarchical Coding - Strict formatting is enforced on identification codes to mirror the database structure. Major codes (`XXX-000`) serve as prefixes for Course codes, which in turn prefix Group codes (`XXX-000-XXX-000-XXX-000`).
- Time Limits - The `working_hours` constraint strictly limits class scheduling to the 08:00 to 20:00 window, rejecting any group assignments that fall outside of standard university operating hours.

9. Limitations

- Bulk Performance: The row-by-row execution of overlap triggers (`trg_group_assignment`) creates significant overhead during large-scale data imports.
- Localization: The `normalize_phone` function defaults to the Polish country code (+48), complicating the management of international student data.
- Scheduling Constraints: The `working_hours` check rigidly enforces an 08:00–20:00 window, and the data model assumes a weekly recurring schedule, limiting support for irregular events.

10. Summary

Database implementation provides a solid foundation for university administration by prioritizing data integrity over flexibility. Through the use of Domain types, Regex constraints, and PL/pgSQL triggers, the system successfully automates error prevention for tasks like scheduling and grading. While the current design involves certain trade-offs regarding performance, it meets the core requirements of standardizing academic records.